

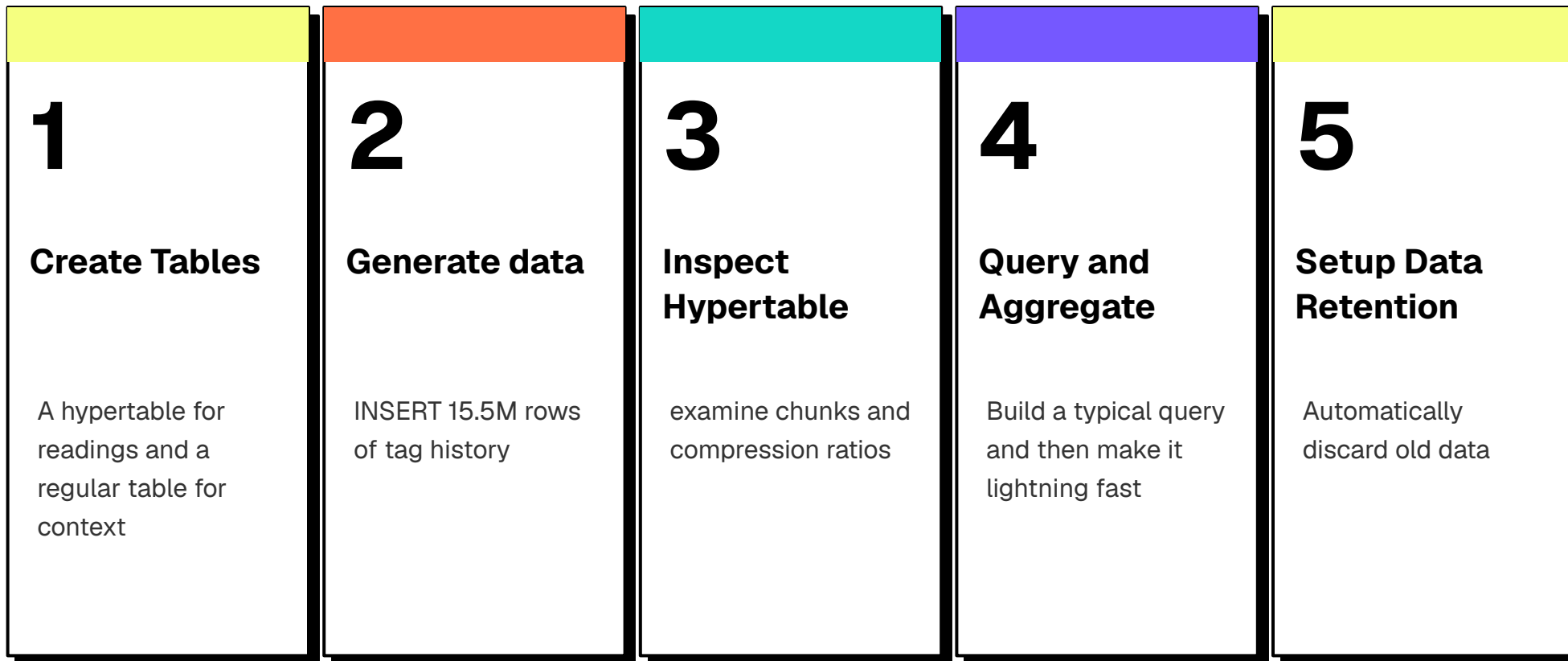


From Sensor to Insight

Real-Time IoT Analytics with TimescaleDB

Here's what we're building

Six steps: From a new Tiger Cloud service to a time-series powerhouse.



Sign up for Tiger Cloud

If you haven't already done so

<https://tsdb.co/sensor-workshop-signup>



All in the docs.

Time-series is its own thing

Same SQL. Same drivers. Time-aware storage and queries.

1

Append-heavy

Mostly INSERTs. Updates and deletes are rare.

2

Time-windowed

Almost every query has a time range filter.

3

Entity-grouped

Data organizes naturally by sensor, device, or user.

The pitch

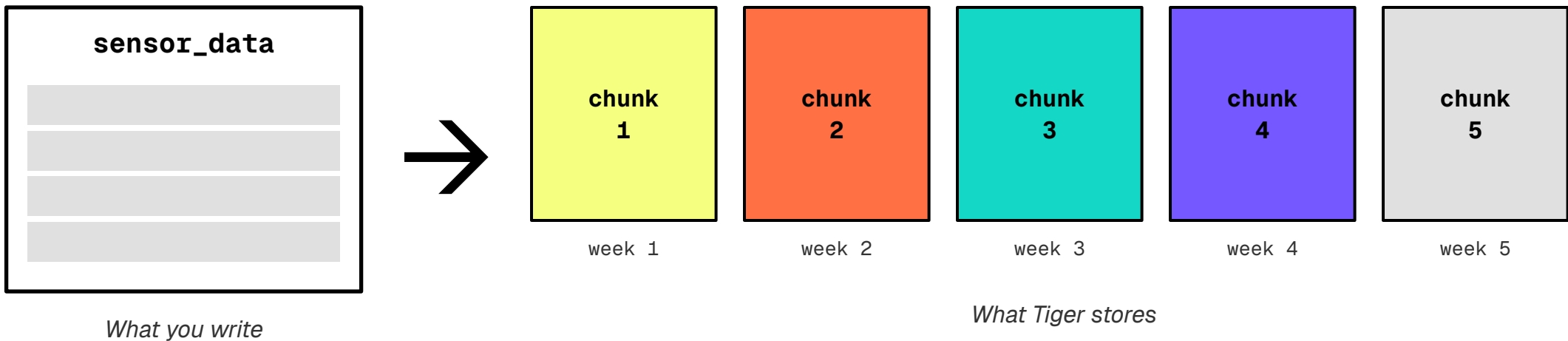
TimescaleDB extends Postgres.

Same SQL. Same drivers. Same ORM.

Under the hood: hypertables, columnar compression, and hyperfunctions.

Hypertables

One table that's secretly many.



Same SQL

CREATE TABLE, INSERT, SELECT all just work.

Auto-partitioned

Tiger picks chunk boundaries. You don't manage them.

Per-chunk ops

Vacuum, compress, drop independently.

Hypertables with Hypercore

Smaller storage. Faster analytics. Same SQL.

Row store · recent chunks

time | tag_id | value | quality

2026-05-20 12:00 | 1 | 22.01 | 192

2026-05-20 13:00 | 1 | 22.11 | 192

2026-05-20 14:00 | 1 | 22.21 | 192

2026-05-20 15:00 | 1 | 22.31 | 192

✓ Cheap writes ✓ Point lookups

Default for recent data.

Columnstore · older chunks

[time1, time2, time3,...]

[tag1, tag1, tag1,...]

[22.11, 22.21, 22.31,...]

[192, 192, 192,...]

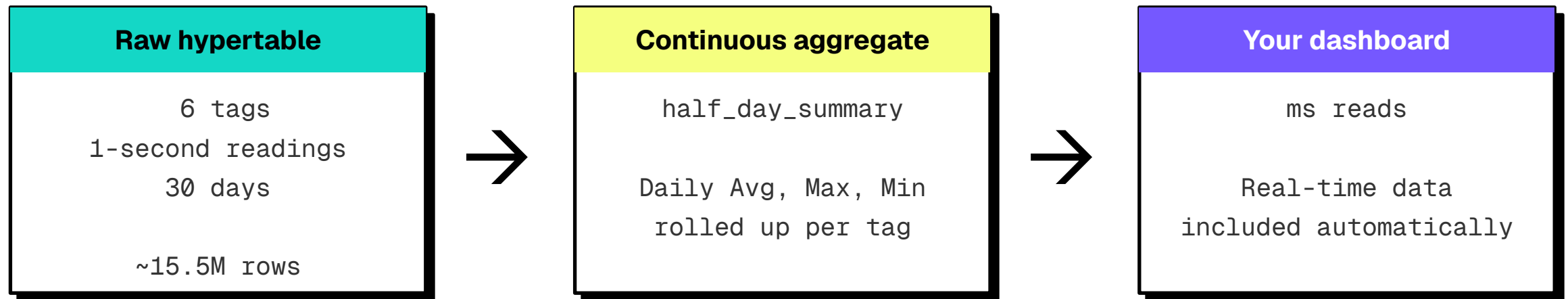
✓ 5–10× smaller ✓ Fast analytics

Sorted, segmented, ready to scan.

Same query. Same SQL. Tiger picks the right storage per chunk.

Continuous aggregates

Pre-computed answers. Always fresh.



Two modes worth knowing

`materialized_only = true`
Only refreshed data is visible.

`materialized_only = false` (real-time)
Recent data shows through transparently.

What you built

In one hour, you have a real working time-series stack.



A hypertable with ~15.5M rows of sensor data



Columnar compression — 5–10× smaller, faster



Typical queries — time buckets, aggregates, joins



A continuous aggregate with real-time updates



A retention policy that drops old chunks

What we skipped

for time —

- Tiered storage to S3
- Sparse index tuning
- Migrating existing tables

All in the docs.

MCP DEMO?





Where to from here?

Keep building

Your service stays up.
Bring your own data.
Plug in Grafana.

`github.com/timescale/
tigerdata-devrel-workshops`

Try the MCP

Tiger MCP works with Claude,
Cursor, Codex, and more.
Install in one command.

`tiger mcp install`

Read the docs

Hypertables, compression,
CAGGs, migrations —
all documented.

`docs.tigerdata.com`

Questions?

Tiered storage

BACKUP

Cold data lives on S3. Queries don't care.

Tier old chunks to S3

`add_tiering_policy('sensor_data', INTERVAL '7 days')` — chunks older than 7d move to object storage.

Query transparently

`enable_tiered_reads` lets a single query span hot + cold storage. No application change needed.

The tradeoff

S3 latency adds milliseconds per chunk read. Great for wide analytical scans, less great for point lookups on cold data.

Sparse indexes

[BACKUP](#)

Tiny metadata. Big skips on compressed scans.

Minmax

Stores the min and max of a column per compressed segment. Skip segments where your range filter can't match.

Bloom

A probabilistic filter for equality lookups. Skip segments that can't contain the value. Great for high-cardinality columns like device IDs.

Configurable

Set `tsdb.sparse_index='minmax(temp), bloom(device_id)'` at CREATE TABLE time, or ALTER later.

Hypercore architecture

How compression actually works: row-by-row → arrays of columns.

Segmentby

Groups data into segments by your chosen key (e.g. sensor_id). Each segment is its own compressed unit.

Orderby

Within a segment, rows are sorted (e.g. time DESC). Sorted-ness is what makes compression effective.

Column-specific algorithms

Gorilla for floats, delta-of-delta for timestamps, dictionary for low-cardinality strings. Tiger picks per column.

Migrating existing Postgres data

BACKUP

You don't have to start from scratch.

New tables

Just CREATE TABLE ... WITH (tsdb.hypertable, ...). Done.

Existing tables

SELECT create_hypertable('mytable', 'time_column', migrate_data => true).

Live migrations

Logical replication + blue/green for zero-downtime cutover. Docs cover the playbook.